EmoBase -- Git Commit & Execution Roadmap

Strategic Engineering Plan: Baseline to Production (Starting Today)

Target Repository: Multimodal Emotion Recognition (Text + Audio + Face + Context Memory)

1. CURRENT BASELINE & ARCHITECTURE STATE (COMPLETED TODAY)

Current Repository Health Status

- Repository Cleaned: Removed 2.73 GB of Google Drive temporary cache (.tmp.driveupload).
- Jupyter Notebooks Replaced: All working notebooks converted to modular Python modules in src/.
- Git Database Optimized: Total Git repository footprint is under 2.0 MB (ultra-fast clone/push).
- Initial Commits Done: 6 atomic commits established covering setup, datasets, extractors, models & docs.
- Large Data Safeguards: .gitignore strictly configured against >100MB weights, avoiding GitHub push blocks.

2. STEP-BY-STEP COMMIT ROADMAP FROM TODAY ONWARDS

Chronological plan divided into 5 strategic phases

Day	Type	Standardized Commit Message	Target Files / Scope
Day 1 (Today)	*chore*	publish main branch to GitHub remote	*git push origin main*
Day 1	*docs*	add remote setup instructions and badges	*README.md*
Day 1	*feat(assets)*	add automated Hugging Face weight puller	*scripts/download_assets.py*
Day 2	*test*	add unit tests for cross-attention & GRU memory	*tests/test_models.py*
Day 2	*test*	add synthetic test fixtures for audio & face shapes	*tests/conftest.py*
Day 3	*ci*	create GitHub Actions CI workflow for test matrix	*.github/workflows/ci.yml*
Day 3	*style*	enforce ruff/black code formatting and linting	*pyproject.toml*
Day 4	*feat(wandb)*	integrate Weights & Biases experiment logging	*src/fusion_layer2.py*
Day 4	*feat(train)*	add early stopping and cosine annealing scheduler	*src/fusion_layer2.py*
Day 5	*feat(ablation)*	implement unimodal ablation benchmark script	*scripts/run_ablations.py*
Day 5	*docs*	record unimodal vs multimodal ablation metrics	*results/results.txt*
Day 6	*feat(infer)*	add end-to-end multimodal inference pipeline	*src/infer.py*
Day 7	*feat(demo)*	build interactive Gradio web application	*app.py*
Day 7	*feat(ui)*	add radar chart visualization for 7 emotions	*app.py, requirements.txt*
Day 8	*build(docker)*	add multi-stage Dockerfile and docker-compose	*Dockerfile, .dockerignore*
Day 9	*docs*	complete comprehensive API docs and video guide	*docs/API.md, README.md*
Day 10	*release*	tag v1.0.0 release and attach binary checkpoints	*CHANGELOG.md, Git Tag v1.0.0*

3. TECHNICAL STRATEGY: LARGE DATA & MODEL WEIGHTS HANDLING

Standard Machine Learning Big Data Protocol

- The 100MB GitHub Boundary: Any file >100MB is automatically rejected by GitHub servers.
- Decoupled Storage: Keep code in Git (<5MB) and host heavy checkpoints (360-500MB) externally.
- Option A (Hugging Face Hub): Free, limitless model hosting; clone weights via snapshot_download().
- Option B (GitHub Releases): Up to 2.0 GB per asset; upload model.safetensors directly to v1.0.0 release tag.
- Option C (Git LFS): git lfs track '*.safetensors' (note: GitHub free tier limited to 2GB total LFS bandwidth).
- Automated Onboarding: Anyone cloning your repo runs 'python scripts/download_assets.py' to get weights.

1. feat: A new feature or capability (e.g., feat(models): implement cross attention block)
2. fix: A bug fix (e.g., fix(audio): resolve sample rate mismatch in librosa loader)
3. refactor: Code restructuring without changing functional behavior (e.g., refactor: convert ipynb to py)
4. docs: Documentation updates only (e.g., docs: update benchmark metrics in README)
5. test: Adding or correcting tests (e.g., test: add test case for tensor batch alignment)
6. chore: Tooling, dependency, or configuration updates (e.g., chore: bump transformers dependency)
7. ci: Continuous integration workflows (e.g., ci: configure GitHub Actions test runner)

Immediate Action Checklist For Today

- Step 1: Create an empty repository on GitHub named 'EmoBase' (do not initialize with README).
- Step 2: Link your remote: git remote add origin https://github.com/Vaibhav-801/EmoBase.git
- Step 3: Push all 6 clean commits: git branch -M main && git push -u origin main
- Step 4: Verify your clean repository is live with the full README and documentation.